Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

TDS Insured England & Wales Deposit Administration API

Document Revision Number: v1.7

API Version Number: v1.4

Date: 01/08/2024

Confidentiality & Notices

This document is the property of The Dispute Service Ltd, whose proprietary rights are included in the information disclosed herein.
The recipient, by accepting this document, agrees that neither this document, nor the information disclosed herein, nor any part thereof shall be reproduced by any means graphic, electronic or mechanical, including photocopying, recording, taping or in information storage and retrieval systems or transferred to other documents or used or disclosed to others for manufacturing or for any other purpose except as specifically authorised in writing by The Dispute Service Ltd.

Copyright 2020 the Dispute Service Ltd. All Rights Reserved
All trademarks and commercially protected names and terms used or referred to within this document are hereby acknowledged as belonging to, and being the property of, their respective owners.

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

Table of contents

1	Document Control	3
1.1	Revision history	3
1.2	Confidentiality	3
1.3	Notices	3
1.4	Document Clarification	3
2	Introduction	4
3	Authentication Information	5
3.1	Client Registration	5
3.2	Client Authorisation	6
4	Deposit Protection/Tenancy Creation	8
4.1	Updating Deposit Protection/Tenancy	9
4.2	Deposit protection/Tenancy Object	10
4.3	Person Object	11
4.4	Response	12
4.5	Ending Deposit Protection/Tenancy	13
5	Creation Status	15
5.1	Request	16
5.2	Response	16
6	API Error Codes	18
7	API URLs	18
8	Additional Functionality	19
8.1	List Branches	19
8.2	Certificate Request	20
8.3	Transferring Deposit Protections	21
8.4	Landlord Search	21

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

1.2 Revision History

Revision Number	Author	Reason for change	
0.9	Lee Perring	Ready for issuing to test partners to test API in sandbox environment	March 2019
0.9.1	Lee Perring, Olisa Onukwa	Added in scope requirement for Oauth2 Process for restricting access to specific functions/scope.	March 2019
1.0	Andrew Owusu & Jorden Adams	Updated document to ensure validation requirements match production	16 August 2019
1.1	Andrew Owusu	Updated document with the correct API version = 1.0 & person object with more details on landlords	11 September 2019
1.2	Andrew Owusu	Updated section 4.1 - updating a Deposit Protection (tenancy)	18 September 2019
1.3	Amanda Portman	Updated document version number, API version number, telephone number and re-ordered 4.1	16 September 2020
1.4	Lee Perring	Updated document version number, API version 1.2 added. Landlord search feature now available under v1.2	08 February 2021
1.5	Lee Perring	Updated document version number, API version 1.4 added. Tenancy lookup feature now available under v1.4	19 July 2023
1.6	Lee Perring	Updated document version number, documented newly required User-Agent header to all requests - **Breaking Change**	01 August 2024
1.7	Lee Perring	Updated document version number, added clarification that rent_amount is a monthly value not weekly	16 August 2024

1.3 Document Clarification

Any questions or queries regarding this report should be addressed to:

Address: The Dispute Service Ltd
West Wing First Floor,
The Maylands Building
200 Maylands Avenue
Hemel Hempstead

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

Herts
HP2 7TG

Phone: +44 (0) 1442 218032

Email : api_enquiries@tenancydepositscheme.com

2. Introduction

Purpose:

This document defines the API provided by TDS (API Insured) to 3rd party platform providers for Agents & Landlords.

Scope:

This document is intended to describe the API interface (messaging formats only) to the TDS Insured system to create new tenancy deposit protection records or edit existing tenancy deposit protection records in a programmatic manner.

A RESTful service has been developed using JSON as a data format.

Access to the API is managed via a token system based on the Oauth2 standard. Software engineers will need to create an account via the TDS website and then generate their access tokens / IDs which can then be used by their application to request access for other user accounts.

Audience:

This document is written for technical suppliers implementing clients of this API.

API Request Header Requirements:

All requests to the API must adhere to the following header requirements:

User-Agent Header:

- The **"User-Agent"** header is mandatory.
- You can choose the value of the User-Agent, but it must be included in every request.
- Requests without a User-Agent header will be rejected.

Prohibited Header:

- The "**x-forwarded-for header**" header is strictly prohibited.

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

- Requests containing the x-forwarded-for header will be rejected for security reasons.
- Ensure that this header is not included in any requests to avoid rejection.

API Processing

Tenancy deposits can be created and then edited using the API provided the deposit is in a state that allows modifications, i.e. the protection has not been ended.

The creation and update process is asynchronous and so once the create or update request has been sent you will then need to send a separate request to find out the status of this initial request, for example you send a create deposit protection/tenancy request you would then be provided an API reference (providing it was successful initially), you'll then 'poll' the API at /status/your_api_reference to get an update on the request to find out when it's been completed and if it was completed without error.

It can take some time for the queue to be processed depending on load and usage and so the client should continue to poll for updates until a completed status is returned.

The service allows for the creation of new tenancy deposit protection records that will be added to a TDS member's account and be protected immediately, in line with the current SFTP service and service provided when registering deposits on the TDS insured website.

Note: *Throughout this document we will refer to 'deposit protections', however to preserve historic links and make integration easier for current SFTP users we have retained the terminology of tenancy as some field labels and within the request points.*

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

API URLs

The URL structure to send calls in the live environment is:

https://api.insured.tenancydepositscheme.com

And within the sandbox environment the URL is:

https://sandbox.api.insured.tenancydepositscheme.com

A version number is required in the URLs as to ensure stability for current systems while the API changes over time.

The current version is set to v1.4

Example: <scheme_url>/v1.4/

For consistency urls will be noted in the document as such:

<scheme_url>/<api_version>/<api_request_point>

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

3. Authentication information - Insured Scheme

The authentication method for the API is Oauth2.

The authentication flow is as follows:

1. Client sends authorisation code request
2. Resource owner allows authorisation request
3. Client is sent an authorisation token
4. Client sends authorisation token and their identifiers to convert to access token and refresh token
5. Client is provided with access token and refresh token provided the auth token is valid for them
6. Client now uses access token for further API calls relating to the resource owner
7. If access token expires, client sends refresh token to receive a new access token

3.1 Client Registration

Software engineers can register for a developer account via the website using the link:

https://api.insured.tenancydepositscheme.com/api-registration/

This allows a user to create an account with developer privileges, form data for their account hasn't been decided but it should be enough to allow communication with developer if needed by TDS so an example list for now is:

- First name
- Surname
- Company Name
- Phone
- Email Address
- Password

Once their account is created and they're logged in they should be able to go to

<scheme_url>/<api_version>/api-account

Here they should be able to view the list of their credentials which is client id, client secret and the redirect URI for them. They should also be able to generate a new set of keys with a redirect URI of their choosing, the client id and secret should be randomly generated and secured.

You will also have the option to set which 'scopes' your client credentials have access to, this is for managing the amount of access you can have against a members account. Currently all API functions

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

are routed under the 'tenancy_management' scope, in future when more features there will be more scope options. The scope you define against your client credentials will then be required to be sent in the client authorisation phase, this means for example if you have tenancy_management as the scope you will need to send &scope=tenancy_management as part of the request, if there is more than one scope then it is a space separated list e.g. & scope=tenancy_management second_scope third_scope.

3.2 Client Authorisation

After the client id has been generated, it can be used for the application (client) to authenticate a user with it.

You should send the user to <scheme_url>/<api_version>/authorize with the below parameter set, response_type, client_id and state.

Example structure :

<scheme_url>/<api_version>/authorize/?response_type=code&client_id=your_client_id&state=random_string&scope=scope_defined_in_previous_step

You must pass the response_type of "code" and then your client id provided in the registration phase. The state parameter is used for you to protect against CSRF attacks (Cross-Site Request Forgery), you should generate a random secure string that is regenerated so this is submitted with the request and on the response you can confirm that this state still exists in the redirect URL.

You must also pass in &scope= as part of the url string, based on what scopes you selected in the generation of your client credentials you will be restricted to those 'scopes' that can be requested, these scopes are space separated in the event you need to use more than 1 scope then please submit these as a space separated string. If the scopes provided do not match what you originally requested via your client credentials step then you will receive an error response stating invalid scope has been provided.

Once the user has been taken to the above URL they will be presented with a login page for their TDS account, once successful they will be prompted to authorize the client to access their account, providing they allow the request they will be redirected to the redirect URI that's associated with the client id, this will also contain the authorisation code and the state (to confirm against CSFR). This code only lasts 30 seconds as it's a short life code to improve security.

After the code has been received you must send a post request to <scheme_url>/<api_version>/token with the username of the client id and client secret as the authentication credentials and for the posted values of

'grant_type' => 'authorization_code',
'code' => 'authorisation_code_from_previous_step'

Curl example:

curl -u client_id:client_secret <scheme_url>/<api_version>/token -d 'grant_type=authorization_code&code=authorisation_code_from_previous_step'

On successful request this will return a json response with the following information:

```
{
    "access_token":"2YotnFZFEjr1zCsicMWpAA",
    "expires_in":3600,
    "token_type": "bearer",
    "scope":"tenancy_management",
    "refresh_token":"tGzv3JOkF0XG5Qx2TlKWIA",
}
```

The access token lasts for 60 minutes (subject to change so rely on the expires_in value), the reason access tokens expire is to ensure if an access code ever gets compromised it has a limited action time. The refresh token is important to keep safe as this will be used to 'refresh' the access token when the access token expires. The expiry time of the refresh token is generally very large or infinite (it usually has to be manually expired by the user), this is to stop the need for the end user to constantly reauthorise the applications access and gives a consistent method for the application to gain access.

If the access token expires then you need to do a curl request to generate a new one which would be done via:

curl -u client_id:client_secret <scheme_url>/<api_version>/token -d 'grant_type=refresh_token&refresh_token=refresh_token_value'

If successful you will receive the json response of

```
{
   "Access_token":"03807cb390319329bdf6c777d4dfae9c0d3b3c35",
   "Expires_in":3600,
   "Token_type":"bearer",
   "Scope":"tenancy_management"
}
```

Once you have an access token you will then simply need to include the access token in subsequent requests as part of the authorization header

Authorization: Bearer access_token example

```
Authorization: Bearer 03807cb390319329bdf6c777d4dfae9c0d3b3c35
{
...
}
```

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

Which will be validated as part of the request to then allow you to continue with the API call such as adding tenancies.

If an access token used is invalid you will get a http 401 error as auth failed.

To check the access token is correct without actually doing any process simply send a request to:

<scheme_url>/<api_version>/verify-access-code

With the Authorization: Bearer access_token set to the http header, if the access code is valid you will receive a response of

```
{
    "success":"true"
}
```

4 Deposit protection (tenancy) creation

The request to create a new deposit protection (tenancy) contains the following meta information:

- Authentication and identification credentials
- Deposit protection/tenancy object
- Person objects(s)

Authentication information

At the start of the data object information must be passed which authenticates and identifies the member and optionally branch to which the tenancies should be appended.

Key	Example	Mandatory	Description	Format Definition
member_id	188832	Yes	The member_id as supplied by the TDS Insured system	Must be 1 - 6 alphanumeric characters
branch_id	188832	Yes	The branch_id as supplied by TDS Insured system.	Must be 1 - 6 alphanumeric characters

Deposit Protection (tenancy) object

A deposit protection/tenancy object contains all the information required to create a single deposit protection/tenancy.

A deposit protection/tenancy contains top level information about the tenancy such as:

- Property details
- Rent information
- 2 or more people records to define all the individuals related to a tenancy.

There must be at least one tenant and one landlord for each deposit protection (tenancy).

If there are 3 tenants and a landlord, 4 person objects would be required. Person objects are defined later in this document.

A request to create a new tenancy contains some meta information about the request, essentially the authentication and identification credentials, one tenancy object and then 2 or more people objects.

A POST request must be crafted and sent to: <scheme_url>/<api_version>/**tenancy**

Alternatively, you can send a POST request to :<scheme_url>/<api_version>/**tenancy/create**

Key	Example	Mandatory	Description	Format Definition
tenancy_id	1	Yes	Your unique reference to this tenancy/deposit protection. If a property with this id already exists within the TDS database, you will edit the existing deposit protection.	
property_paon	1 or The Corner House	Yes	This is the "building name" or "building number" of the rented property. In essence the house number or name.	Must be between 1 and 100 alphanumeric characters.
property_saon	"Flat 2" or "Room 2"	No	This is only required when the "building name" or "building number" (property_paon) is insufficient for example 'Flat 2'.	Must be between 1 and 100 alphanumeric characters.
property_street	The Street	No	The street where the rented property is located.	Must be between 1 and 100 alphanumeric characters.
property_locality	Test Locality	No	Locality where rented property is located (suburb, parish, industrial estate).	Must be between 3 and 100 alphanumeric characters.
property_town	Test Town	No	Town where the rented property is located.	Must be between 3 and 100 alphanumeric characters
property_administrative_area	Test Admin Area	No	Administrative area where the rented property is located.	Must be between 3 and 100 alphanumeric, period and hyphen characters.
property_postcode	AB11CD	Yes	Postcode of rented property.	Must be between 1 and 8 alphanumeric characters. Is a valid England and Wales postcode. Spaces are not accepted.
tenancy_start_date	01/11/2016	Yes	Start date of tenancy as per tenancy agreement	Format must be DD/MM/YYYY: Must be a date between 01/01/1980 and 31/12/2099
tenancy_expected_end_date	31/10/2017	No	Expected end date of tenancy as per the tenancy agreement	Format must be DD/MM/YYYY: Must be a date between 01/01/1980 and 31/12/2099. The value for the tenancy_expected_end_date must falls after the tenancy_start_date
rent_amount	999	Yes	Rent amount payable as per the tenancy agreement. **This should be the monthly rent amount, not weekly.**	Must be between 0.00 and 999999.99.

deposit_amount	1250	Yes	Full deposit amount as per tenancy agreement.	Must be between 0.00 and 999999.99.
deposit_received_date	31/10/2016	Yes	Date that deposit was received by the member.	Must be a valid date between 01/01/1980 and 31/12/2099.
number_of_tenants	1	Yes	Total number of tenants listed on tenancy agreement.	Must be between 1 and 99.

Person object

Each person associated with a deposit protection (tenancy) is defined in a separate object which are collectively provided to the API as an array of objects.

Note: LL = Landlords, TT = Tenant;

Key	Example	Mandatory	Description	Format Definition
person_classification	Landlord	Yes	This field is used to classify the type of party associated with the deposit protection. Each deposit protection (tenancy) must have at least one landlord and one tenant. The Insured API only works with 1 landlord as a tenancy can only be under 1 landlord e.g. the main landlord.	Supported values are: • Landlord • Tenant
person _id	123456	Yes	If the "person_id" i.e. landlord or tenant already exists within the TDS Insured system you can provide their ID here to ensure the deposit protection (tenancy) is associated with the existing record. ***Note:*** *all other details provided must match the existing record too (e.g. member/branch/landlord and tenancy ID).*	Existing person ID (1 - 100 alphanumeric characters)
person_reference	LL123ABZ	No	A user given reference for this person (only used where person type is landlord)	
person _title	Mr	Yes	Title of the person, e.g. Mr, Mrs, Miss etc.	Must be between 1 and 30 characters.
person _firstname	John	Yes & No, dependent on person type	First name of the person. • For landlords, this is required if the surname information is blank • For tenants the first name is required	Must be between 2 and 255 characters.
person _surname	Calder	Yes & No, dependent on person type	Surname of the person. • For landlords, the surname is required if the first name is blank • For tenants, the surname is required	Must be between 1 and 255 characters.

person_paon	1 or "The Corner House"	Yes	Used for landlord address only. The primary address or way of describing the building name or number of the tenancy address, Typically contains the house name or number.	Must be between 1 and 100 alphanumeric characters.
person_saon	"Flat 2" or "Room 2"	No	Used for landlord address only. Secondary address or way of describing the building name or number, Only used when the name or number (PAON) is not enough, for example 'Flat 2'.	Must be between 1 and 100 alphanumeric characters.
person_street	The Street	Yes	Used for landlord address only. Street where landlord's property is located.	Must be between 1 and 100 alphanumeric characters.
person_locality	Camden	No	Used for landlord address only. Locality where landlord's property is located. Locality where property is located.	Must be between 3 and 100 alphanumeric characters.
person_town	T	Yes	Used for landlord address only. Town where landlord's property is located.	Must be between 3 and 100 alphanumeric characters.
person_administrative_area	Test Admin Area	No	Used for landlord address only. Administrative area where landlord's property is located.	Must be between 3 and 100 alphanumeric characters.
person_postcode	AB11CD	Yes	Used for landlord address only. Postcode of landlord's property.	Must be between 1 and 8 characters.
person_country	United Kingdom	Yes	Used for landlord address only. Country where landlord lives.	Must be between 1 and 100 alphanumeric characters.
person_email	tenant1@test.com	Yes & No, dependent on person type	e-mail address of the landlord or tenant. • For landlords the email address is not required (mandatory) • For a tenant the email address is required (mandatory) if a mobile number has not been provided	Must be between 1 to 255 characters. Must be unique, 2 parties cannot share an email address. Must have a valid email syntax and domain
person_mobile	07717 123456	Yes & No, dependent on person type	Mobile phone number of the landlord or tenant. • For landlords the mobile number is not required (mandatory) • For a tenant the mobile number is required (mandatory) if an email address has not been provided	Must be between 1 and 30 characters including area codes and if not in the UK, any international dialling codes. Must start with a 0 or + character. Where person type is tenant - either mobile or e-mail are mandatory not both.

The overall abridged data structure is indicated below:

```
{
    "member_id":"18881",
    "branch_id":"20001",
    "scheme_type": "Insured",
    "tenancy":{
        "property_id":1,
        "property_paon":"1",
```

```
        "property_saon":"",
        "property_street":"Test Street",
….
        "number_of_tenants":1,
        "number_of_landlords":1,
        "people":[{
            "person_classification":"Lead Tenant",
            "person_id":"",
            "person_reference":"",
            "person_title":"Mr",
            "person_firstname":"Lead",
            "person_surname":"Tenant",
            "is_business":"N",
….
            "person_correspondence_country":"United Kingdom"
        },{
            "person_classification":"Primary Landlord",
            "person_id":"",
            "person_reference":"",
            "person_title":"Miss",
            "person_firstname":"Primary",
            "person_surame":"Landlord",
            "is_business":"N",
….
            "person_correspondence_country":"United Kingdom"
        }]
    }
}
```

For a full list of fields please see section Tenancy Object

4.1 Update request - Deposit protection (tenancy)

To update a tenancy in the insured scheme you can send your request to **<scheme_url>/<api_version>/tenancy/update/dan**

Alternatively you can submit a request exactly to the creation process (**/tenancy**) and providing you use the same ids as used in the original creation step and the deposit protection/tenancy is in an editable state (not paid for, not protected and not ended), any fields you then submit will be checked and updated accordingly.

This process does not require people to be sent, so if you only need to amend details on the tenancy/property (not tenants/landlord) then you can remove them from the request and simply send the fields you need to change such as "property_paon", an example update request is shown below:

```
{
    "member_id":"18881",
```

```
    "branch_id":"20001",
    "scheme_type": "Insured",
    "tenancy":{
        "property_id":1,
        "property_paon":"21",
        "property_saon":"",
        "property_street":"Second Street"
     }
}
```

A POST or PUT request must be crafted and sent to:

<scheme_url>/<api_version>/**tenancy/update/dan**

Alternatively you can send the request to :<scheme_url>/<api_version>/**tenancy/create** as explained in section 4.

4.2 Response

When a deposit protection/tenancy creation request as defined above is submitted the service will return a response with the following information.

Key	Example	Mandatory	Description
success	true	Yes	Either true if successfully submitted for processing by the tenancy creation engine, or false if an error was detected in the initial request. **Note:** This flag only reports the success of the request and in no way indicates that the tenancy has been created. This information can only be determined with a follow-up call to the CreationStatus service, detailed below.
api_reference	167237	No	If success is true and the requested tenancy has been posted for creation then this will return the unique id for this request. This reference is required for all subsequent calls to CreationStatus in order to determine the outcome of the tenancy creation process.
error	Fail	No	If success is false this is a text representation of the pre-processing error which caused prevented the tenancy creation from being submitted for processing

Example response:

```
{
    "success":"true",
    "api_reference":"167237",
```

```
{
    "Success":"false",

 errors:[
    {
        error:"Not authorised to make amendments to this member",
        error_code:1234
```

If the error is for invalid authorisation due to invalid token it should return a 401 http error with some extra code (to be added to the joint sheet document for shared codes) to provide info on auth error.

4.3 Ending Deposit protection (tenancy)

To end a tenancy, you must submit a request to the tenancy/update endpoint as you are 'updating' the tenancy.

On this request you would need to populate the field **'end_type'** with a value of **'renewed', 'periodic', 'no_dispute', 'dispute', 'transfer'.** The below tables should you any other supporting fields that are required based on the type of end_type you are setting.

Renewed: The tenancy has been renewed. If this is valid and updated the field **'expected_end_date'** should be provided else it will remain unchanged.

Field Name	Value Format	Requirements
expected_end_date	dd/mm/yyyy	Optional

Periodic: The tenancy has become periodic (active but not removed, past its original tenancy date). If this is valid and updated the field **'expected_end_date'** should be provided else it will remain unchanged.

Field Name	Value Format	Requirements
expected_end_date	dd/mm/yyyy	Optional

No Dispute: The tenancy has ended without dispute. You must provide the field **'end_date',** you must also provide as much information as possible to the fields listed under no_dispute, however these are optional.

Field Name	Value Format	Requirements
end_date	dd/mm/yyyy	Mandatory
deposit_protection_end_date	dd/mm/yyyy	Optional
deposit_amount_returned	Decimal 2DP	Optional
paid_landlord	Decimal 2DP	Optional
paid_landlord_date	dd/mm/yyyy	Optional
paid_agent	Decimal 2DP	Optional
paid_agent_date	dd/mm/yyyy	Optional
paid_tenant	Decimal 2DP	Optional
paid_tenant_date	dd/mm/yyyy	Optional

Dispute: The tenancy has ended in dispute. You must provide the field **'end_date'. Marking the tenancy has ended in dispute does not start the dispute process at TDS, to do this you must visit the TDS website and proceed through the dispute process as normal.**

Field Name	Value Format	Requirements
end_date	dd/mm/yyyy	Mandatory

Transfer: The deposit protection/tenancy has been transferred to another scheme. You must provide the field **'end_date'.**

Field Name	Value Format	Requirements
end_date	dd/mm/yyyy	Mandatory

Status Response Structure

The response for the ending of the deposit protection/tenancy will be identical to the updating section, refer to this for the correct structure to anticipate.

Updated successfully:

```
{
    success:"true",
```

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

```
    status:"updated",
    dan:"NI0000123",
 }
```

Failed Update:

```
{
    success:"true",
    status:"failed",
    dan:"NI0000123",
    errors:[
    {
        record:"tenancy",
        record_id:"1",
        field:"end_date",
        error:"Please ensure the end_date field is a valid date",
        error_code:72011
    },
    {}
    ]
```

5. Creation Status

Once a deposit protection/tenancy creation requested has been posted to the service, if there are no problems found, the deposit protection/tenancy will be created within the TDS account. The deposit protection/tenancy will also be visible via the website, however, in order to check on the status of the deposit protection/tenancy creation and retrieve information about any errors found, calls can be made to the CreationStatus service.

5.1 Request

Request a status update by sending a request to**:**
<scheme_url>/<api_version>**/status/your_api_reference**

Example:

<scheme_url>/<api_version>/status/167237

The access token to authenticate and the value of the api_reference that was provided by the initial response, this will then send a response as laid out in the following section.

5.2 Response

The response object will always return a success element which will be true if the request was successful. This does not in anyway indicate that the deposit protection/tenancy creation worked, but just that the api request to retrieve the state was successful. If the success status is false then an error object will be present too which will provide a reason for the failure, such as bad authentication credentials for the api_reference provided. The status element can be one of three values, "pending", "created" and "failed" and indicates whether the deposit protection/tenancy creation request is still awaiting processing, has been successful or failed for some reason. Successful creation items will return a "dan" element, which will provide the identification reference for the newly created tenancy.

In the event of the status being failed there will be an errors array returned. This will contain an element for each field which generated an error. Each error should always contain an error and error_code element and in most scenarios it should contain record and record_id.

The key is the field name and the value is the error string. A successful creation and error object can also return a warnings array, issues for consideration, but which would not in themselves caused the deposit protection/tenancy to not be created.

Key	Example	Mandatory	Description
success	true	Yes	Either true if successfully submitted for processing by the tenancy creation engine, or false if an error was detected in the initial request. **Note:** This flag only reports the success of the request and in no way indicates that the tenancy has been created.
status	created	Yes	Can be "created", "failed" or "pending"
dan	NI0000123	No	The reference of the new tenancy if status was success. Not present if pending or failed status.
errors	array	No	An array of field:reason elements if any errors found. The status will be failed and errors will be present. This element will not be present for created and pending statuses.

Examples:

Submitted, operation still pending:

```
{
    success:true,
    status:"pending",
}
```

Process failed due to errors, not created:


```
{

    success:"false",
    status:"failed",
    dan:"",
    errors:[
    {
        record:"tenancy",
        record_id:"1",
        field:"property_postcode",
        error:"Please ensure the Postcode field conforms to  BS7666:2000",
        error_code:34011
    },
    {
        record:"person",
        record_id:"1",
        field:"person_phone",
        error:"Please ensure the Phone field is a valid phone number",
        error_code:33101
    },

    ]
    warnings:[
    {
        record:"tenancy",
        record_id:"1",
        field:"property_paon",
        error:"Please ensure the Paon field has been completed",
        error_code:34011
    },
    {}
```


Errors are returned as an array of separate errors. Where possible the errors/warnings should also include the 'record' which is the type e.g. tenancy, person and their record_id which is the id that was used in the request so when submitting a tenancy with the property_id of 1 this will be used in the record_id field as this will aid in rectifying issues when for example you add 10 tenants but only 5 have a phone number that is invalid.

6. API Error Codes

Error codes are provided in most error responses where possible, for more information on what the error code means you can access it in the browser via:

<scheme_url>/error-codes or by going to: <scheme_url>/error-codes

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

7. API URLs

URL	Description
https://api.insured.tenancydepositscheme.com/<api_version>/api-home/	Location for developer to create and administer their account to setup client ids for use in their application.
https://api.insured.tenancydepositscheme.com/<api_version>/authorize/	Part of the Oauth process, used in the third parties' API to send user to this page to grant the software access to the users account.
https://api.insured.tenancydepositscheme.com/<api_version>/token	Part of Oauth process, converts auth token or refresh token into access token as described in the Authentication section.
https://api.insured.tenancydepositscheme.com/<api_version>/verify-access-code	Validates if the access code provided is valid, provides a quick means of checking &Oauth has been setup correctly.

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

https://api.insured.tenancydepositscheme.com/<api_version>/tenancy	Send tenancy creation or update request to this location with the tenancy details.
https://api.insured.tenancydepositscheme.com/<api_version>/status	Provides the status response of a previous API request
https://api.insured.tenancydepositscheme.com/<api_version>/branches/<member_id>	Provides a list of branches available within a member's account.
https://api.insured.tenancydepositscheme.com/<api_version>/certificate/<deposit_UID/DAN>	Provides a link to download the deposit protection certificate of the specified deposit protection.
https://api.insured.tenancydepositscheme.com/<api_version>/transfer/<deposit_UID/DAN>	Send a transfer request to move a deposit protection to a different branch and/or landlord.
https://api.insured.tenancydepositscheme.com/<api_version>/search/landlord	Search for specific landlords within a given members account
https://api.insured.tenancydepositscheme.com/<api_version>/tenancy/lookup/<deposit_UID/DAN>	Search for a specific tenancy via its DAN (unique id) to see the current registered details of the landlord, tenants and the tenancy itself.

8 Additional Functionality

8.1 List Branches

As members may already have branches registered with the TDS system the facility to get the branches associated with the member is provided to assist with management.

To get the list of branches send request to:
<scheme_url>/<api_version>/branches/<member_id>

Example:

https://api.insured.tenancydepositscheme.com/v0.9/branches/188832

This will return a list of branches with their details in the following format:


```
branches:[
     20001: {
        "member_id":"18881",
        "branch_id":"20001",
        "branch_name": "Plymouth Branch",
        "branch_paon": "1",
        "branch_saon": "",
        "branch_street": "Test Street",
        "branch_town": "Plymouth",
        "branch_administrative_area": "",
        "branch_locality": "",
        "branch_postcode": "PL9 7BL",
        "branch_phone": "07722886421",
        "branch_email": "branch@email.com",
        "branch_contact_name": "Lee Perring",
        "branch_status": "Open"

      },
     20002: {
        "member_id":"18881",
        "branch_id":"20002",
        "branch_name": "London Branch",
        "branch_paon": "2",
        "branch_saon": "",
        "branch_street": "Street",
        "branch_town": "London",
        "branch_administrative_area": "",
        "branch_locality": "",
        "branch_postcode": "PL6 8BX",
        "branch_phone": "07722664855",
        "branch_email": "londonbranch@email.com",
        "branch_contact_name": "John Doe",
        "branch_status": "open"
```


So each branch is returned in a branches array with their branch id as the array key so you can quickly reference a specific one based on your own ids e.g. get all the arrays then check branch[20002] exists rather than having to iterate through all the time.

Branches can have their status changed by TDS as this is currently handled by them when a member wants to close a branch and for any other reason. This status is provided in the response so you can

Docusign Envelope ID: D55BD495-F012-4F69-A5C6-7828E0AC5A7E

handle accordingly. The functionality to close branches may be opened up in the API but as of v0.9 this is not possible. **The possible options for branch status are simply "open" or "closed" so you only need to cater for these two values, if a branch is closed you cannot add tenancies to it.**

8.2 Deposit Protection certificate request (link to pdf file)

This request obtains the deposit protection certificate by sending a request to:

<scheme_url>/<api_version>/certificate/DAN

For the insured scheme the DAN would be like:
<scheme_url>/<api_version>/certificate/NGGYU

This should provide a link to the pdf download of the tenancy certificate.

An example response:

```
{
    "success":"true",

"certificate":"https://tenancydepositscheme.com/member_generate_tenancy_pdf.php?t=NGGYU",
}
```

While this url is easy to generate (it's simply the t-value that changes) this location is always subject to change so it is strongly advised to use the /certificate/ call as this means you will always get the correct URL to obtain the deposit protection certificate.

8.3 Transferring Deposit protections (tenancies)

In order to transfer deposit protections you can send a request to:
<scheme_url>/<api_version>/transfer/[Tenancy UID/DAN]

For example the request would be:
<scheme_url>/<api_version>/transfer/NGGYU

In order to then transfer the given deposit protection you must provide at least a branch_id to transfer to.

The following parameters can be sent:

```
{
    branch_id:"2001",
    person_id:"123",
```

Branch ID must be provided as this is used as the target location, if no person_id is provided then the system will make a copy of the current landlord on the deposit protection and replicate this on the target branch. If the landlord id is already taken on the target branch then the deposit protection will simply move to this landlord rather than being overwritten.

While we support person_id we do also allow for you to send it as landlord_id instead, they can be used interchangeably.

Status Response Structure

Once the request is completed you will receive feedback of the standard api request received with the request id. You can then submit a status request to the /status/ url and will either receive the pending status of you will receive all the information in regards to the newly transferred tenancy which will look like this:

```
{
    success:"true",
    dan:"NGGYU",
    property_id:"1",
    branch_id:"2001",
    person_id:"123",
```

The return includes the DAN (unique id) just for a reference and as property_ids are only unique per landlord and the person_id is unique per branch there is a possibility that this could be increased to ensure it remains unique in its new location, this is why we pass back what the property id is and the same goes for person_id. As long as you have the DAN then you can always get the tenancy details.

8.4 Landlord Search

In order to search for a landlord you can send a request to:
<scheme_url>/<api_version>/search/landlord

The following parameters can be sent:

```
{
    member_id:1,
    page:1,
```

```
    query:"landlord@email.com",
    archived:false,



}
```

Member ID is the member you want to search within incase of an account with multiple memberships.

Page specifies the page you want returned in the resultset, the default is 1.

Query is for the search term which is a minimum of 3 characters and will search the following fields: 'firstname', 'surname', 'phone', 'email', 'paon', 'saon', 'street', 'locality', 'town', 'administrative_area', 'postcode'.

Search response structure

Once the request is complete the response will be returned as a the below example:

```
{
    current_page:1,
    has_more:true,
    results:[
    {
     person_classification:"Landlord",
     member_id:"12",
     branch_id:"12:23",
     person_id:"456",
     person_firstname:"Bob",
     person_surname:"Turner",
     person_paon:"1",
     person_saon:"",
     person_street:"Bakers road",
     person_locality:"",
     person_town:"Plymouth",
     person_administrative_area:"Devon",
     person_postcode:"PL8 8BQ",
     person_country:"United Kingdom",
     person_email:"email@gmai.com",
     person_fax:"",

    },

    ],
    archived:false,



}
```

Current page – this is the page that the results refer to, by default it is 1.
has_more – this signifies if there are more result pages to view
results – contains array of any landlords found with their data summary within each.
Archived – confirms if search request was for archived or live landlords

8.5 Tenancy Lookup

In order to search for a landlord you can send a request to:
<scheme_url>/<api_version>/tenancy/lookup/[Tenancy UID/DAN]

For example the request would be:
<scheme_url>/<api_version>/tenancy/lookup/NGGYU

When a request has been made the system will return the details of the tenancy if found, this will look like this:


```
{
    "success": "true",
    "api_reference": 30,
    "property_id": "99771270222",
    "dan": "st-8CSTAMaHr",
    "property_paon": "4",
    "property_saon": "Studio 61",
    "property_street": "Robertson Wall",
    "property_locality": "Simpsonmouth",
    "property_town": "Thompsonville",
    "property_administrative_area": "Somerset",
    "property_postcode": "P1 7YT",
    "tenancy_start_date": "25/02/2023",
    "tenancy_expected_end_date": "25/02/2023",
    "rent_amount": "10.00",
    "deposit_amount": "10.00",
    "deposit_received_date": "25/02/2023",
    "number_of_tenants": "1",
    "end_type": "",
    "end_date": "",
    "deposit_protection_end_date": "",
    "deposit_amount_returned": "0.00",
    "paid_landlord": "",
    "paid_landlord_date": "",
    "paid_agent": "",
    "paid_agent_date": "",
    "paid_tenant": "",
    "paid_tenant_date": "",
    "status": "live",
    "people": [
        {
            "person_classification": "Landlord",
            "person_id": "2880658",
            "person_firstname": "Jackson",
```

```
            "person_surname": "Mason",
            "person_paon": "6",
            "person_saon": "Studio 69y",
            "person_street": "Hughes Ranch",
            "person_locality": "East Toby",
            "person_town": "Thomaston",
            "person_administrative_area": "Dumbartonshire",
            "person_postcode": "Y8 8RA",
            "person_country": "United Kingdom",
            "person_email": "harley.smith@example.net",
            "person_mobile": "07258208236",
            "person_fax": ""
        },
        {
            "person_classification": "tenant",
            "person_id": "12845486",
            "person_firstname": "Finley",
            "person_surname": "Phillips",
            "person_email": "gavin24@example.org",
            "person_mobile": "07974402096"
        }
    ]
}
```


If the tenancy cannot be found it will return an error shown below:


```
{
    "api_reference": 31,
    "errors": [
        {
            "error": "Invalid tenancy code provided",
            "error_code": 11010
        }
    ]
}
```